

트러블슈팅 narrative 3건 (발표용)

목적: PPT [26] 슬라이드의 원자료. 5단계(증상→가설→디버깅→해결→학습) 형식으로 정리하고, 학습 부분은 **본인 학습**과 **Pre-Sales 학습**(고객-runbook 관점) 두 갈래로 나눔.

선정된 3건 (Network / Cloud Security / Automation 의 3개 layer 분포):

1. Tailscale ProxyJump → Subnet Router 우회 (네트워크)
2. IAM Bootstrap 역설 (클라우드 보안)
3. Ansible `ansible_user` 누락 (자동화)

표기 규칙 (메모리 feedback 반영):

- 🗨️ = **본인 사고** — 본인이 의심하고, 본인이 확인하고, 본인이 결론 내린 부분
- 🤖 = **AI 보조** — AI 가 가설을 제시하거나 명령어를 알려준 부분
- 🔧 = **검증 명령/로그** — 실제 사용한 명령, 출력 일부

Pre-Sales 톤 원칙: "이 함정을 고객에게 권할 때 어떻게 사전 차단할 것인가" 가 학습의 마지막 줄이 되어야 함.

출처: 04-23 학습 노트 §3 (Tailscale), §6 (Ansible) / 04-23 phase2-success §2-1 (IAM bootstrap). 모두 본인이 당시에 직접 정리한 1차 자료에서 추출.

Narrative 1: Tailscale ProxyJump → Subnet Router 우회 — 5시간 디버깅이 아키텍처 결정으로

1. 증상 (Symptom)

🗨️ Phase 1 첫 빌드 직전, Jenkins 가 온프레임 VM 으로 Ansible 을 던지지 못함.

특이한 점: 각 경로는 독립적으로 OK, 조합만 실패.

경로	결과
Jenkins → HAProxy 직접 SSH	✅ 성공
HAProxy → WEB-WAS 직접 SSH	✅ 성공
Jenkins → HAProxy → WEB-WAS (ProxyJump)	❌ 실패

🔧 예러:

```
kex_exchange_identification: Connection closed by remote host
```

각 노드 독립적으론 정상이라, 인증-네트워크-방화벽 모두 표면적으론 문제 없어 보임.

2. 가설 (Hypothesis)

- 🟡 가설 A: SSH key 등록 문제 → 직접 경로는 OK 라 키는 정상. 기각.
- 🟡 가설 B: 네트워크 자체 문제 → TCP 단계가 정상이어야 ProxyJump 가능, 그런데 직접 SSH 가 통과하므로 TCP 는 정상. 기각.
- 🤖 가설 C: ControlMaster 소켓 꼬임 — Ansible 이 SSH 재사용 위해 만드는 캐시 (/var/lib/jenkins/.ansible/cp/).
- 🤖 가설 D: HAProxy sshd 의 **AllowTcpForwarding** 등 ProxyJump 관련 옵션 비활성화.

3. 디버깅 (Debugging) — 레이어별 격리

🟡 "어디서 막히는지 모르면 레이어를 하나씩 격리한다" 로 접근.

3-1. TCP 레이어



```
nc -vz 192.168.20.12 22
# → Ncat: Connected to 192.168.20.12:22
```

TCP 3-way handshake OK → 네트워크는 정상.

3-2. SSH banner



```
(echo ""; sleep 2) | nc 192.168.20.12 22
# → SSH-2.0-OpenSSH_8.7
```

sshd 자체는 정상 응답.

3-3. sshd 실효 설정 (-T)



```
sudo sshd -T | grep -iE 'allowtcpforwarding|permittty|maxsessions'
# → allowtcpforwarding yes
```

정상. 가설 D 기각.

3-4. ControlMaster 의심 → 검증

🤖 + 🟡 Ansible 의 SSH 재사용 캐시 제거 + 비활성화로 재시도: 🛠️

```
sudo rm -rf /var/lib/jenkins/.ansible/cp/*
ANSIBLE_SSH_ARGS="-o ControlMaster=no -o ControlPath=none" ansible ...
```

여전히 실패. 가설 C 기각.

3-5. -W 수동 ProxyCommand



```
sudo -u jenkins ssh -l ansible -vvv -W '[192.168.20.12]:22' 100.105.181.80
```

출력 분석:

```
channel 0: open confirm rwindow 2097152 rmax 32768    ← 터널 열림
SSH-2.0-OpenSSH_8.7                                  ← WEB-WAS 배너 도달
channel 0: rcvd eof                                    ← 이상하게 EOF
```

💡 터널 레벨은 정상인데 **Ansible** 만 실패 — 더 파도 원인이 보이지 않음. 3시간 누적.

4. 해결 (Resolution) — 디버깅 포기 + 우회로 채택

🤖 AI 가 제안: "디버깅 시간 vs 우회 시간을 비교하라. 30분~1시간 진전 없으면 우회로가 있는지 자문하는 게 엔지니어링 현명함" 이라는 판단 기준 + Tailscale **Subnet Router** 패턴으로 우회하는 구체 방안.

💡 본인 판단: 이미 3시간 누적, AI 제안의 합리성 인정 → 채택 결정.

→ 아키텍처 변경 (10분 작업).

4-1. HAProxy 에서 IP forwarding + 서브넷 광고



```
# IP forwarding 영구 활성화
sudo tee /etc/sysctl.d/99-tailscale.conf > /dev/null <<'EOF'
net.ipv4.ip_forward = 1
net.ipv6.conf.all.forwarding = 1
EOF
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf

# 서브넷 광고
sudo tailscale up \
  --advertise-routes=192.168.20.0/24 \
  --accept-routes \
  --reset
```

4-2. Tailscale Admin Console 에서 route 승인

브라우저: <https://login.tailscale.com/admin/machines> → HAProxy → Edit route settings → 192.168.20.0/24 체크 → Save.

4-3. firewalld masquerade (응답 경로 확보)



```
sudo firewall-cmd --permanent --zone=public --add-masquerade
sudo firewall-cmd --permanent --zone=trusted --add-interface=tailscale0
sudo firewall-cmd --reload
```

4-4. Ansible inventory 에서 ProxyJump 제거

```
# Before
on-prem-webwas:
  ansible_host: 192.168.20.12
  ansible_ssh_common_args: "-o ProxyJump=ansible@100.105.181.80"

# After
on-prem-webwas:
  ansible_host: 192.168.20.12
  # subnet router 직결 - ProxyJump 불필요
```

→ Phase 1 빌드 첫 성공 🎉

5. 학습 (Learning)

5-1. 본인 학습 (Engineering)

- 🌸 **레이어별 독립 검증의 힘**: TCP → SSH banner → ControlMaster → -W 수동 — 각 레이어를 격리해서 테스트할 수 있어야 원인 위치 파악 가능. "막혀있다" 수준의 정보로는 해결 불가능.
- 🚗 → 🌸 **디버깅 시간 vs 우회 시간 비교**: ProxyJump 추적 ~3시간 vs Subnet Router 우회 10분. AI 가 제시한 "30분~1시간 진전 없으면 우회 자문" 판단 기준을 체크 — 다음에 비슷한 상황에서 본인이 먼저 자문하게 됨 (AI 협업의 자산화).
- 🌸 **Tailscale Subnet Router 의 본질**: 단방향 게이트웨이 (Tailscale → 서브넷 inbound only). MASQUERADE 가 응답 경로 확보의 핵심 — "패킷이 갔는데 응답이 없다" 의 진단 단서.
- 🚗 → 🌸 **"왜 안 되는지 완벽히 이해"** 보다 **"되는 길로 돌아가기"** 가 엔지니어링 현명함 — AI 가 가르쳐준 원칙을 본인이 받아들임. 학습은 다음에 계속하면 됨.

5-2. Pre-Sales 학습 (고객/runbook 관점)

고객사에 Tailscale 기반 Hybrid 연결을 권할 때 사전 차단 체크리스트:

1. ProxyJump vs Subnet Router 의사결정 매트릭스 사전 제시

조건	권장 패턴
Jump host 1대 + bastion 패턴 익숙	ProxyJump
백엔드 다수 + agent 못 까는 자원(RDS 등) 포함	Subnet Router
Tailscale 라이선스 비용 압박	Subnet Router

→ 우리 사례는 후자였는데 ProxyJump 로 시작했다가 5시간을 잃음. **사전 매트릭스만 있었으면 0시간.**

2. Subnet router 호스트의 4중 정합 사전 검증

- `sysctl ip_forward = 1` (영구)
- `firewalld tailscale0` 을 trusted zone
- Tailscale Admin Console route approval
- SNAT 또는 MASQUERADE 활성화

3. 각 호스트에 Tailscale 까는 naive 패턴은 RDS 같은 managed service 등장 시 깨짐 — PoC 단계에서 미리 인지하고 처음부터 subnet router 패턴으로.

→ 이 셋이 사전 검증되면, 동일 디버깅 사이클 (5시간) 을 0 으로 단축. 동시에 **Slide [24] HAProxy-only Tailscale 아키텍처의 origin story** 가 됨.

Narrative 2: IAM Bootstrap 역설 — IaC 가 자기 권한을 자기가 관리하는 닭/달걀

1. 증상

🌸 Phase 2 자동화를 위해 Jenkins EC2 가 자체 Terraform 을 실행해야 함. 그런데:

🔧 `terraform init` 자체가 실패:

→ 파이프라인이 시작조차 못 함.

- 가설 A: 단순히 IAM 권한이 부족하다 — 권한 추가하면 끝.
- 가설 B: 그러나 "권한 추가" 만으로 끝나지 않는 구조 — IaC 가 자기 자신의 권한을 관리하므로:

→ laC 자기참조 구조의 닭/달걀. "권한을 줘야 권한을 줄 수 있다"는 패러독스.

- 본인 노트북에 admin IAM User 보유 — **의도적 분리** (운영용 IAM 과 bootstrap 용 IAM 을 처음부터 별도 계정으로 세팅해둔 설계 선택)
- → 이게 그대로 **bootstrap** 경로 역할 가능. 사후가 아닌 **사전 설계**의 결과.

[운영 경로 - 반복]

Jenkins EC2 (위 Sid 들 보유)

↓ terraform apply

운영 자원 변경

7개 Sid 추가 후 Phase 2 파이프라인 정상 동작.

🔒 보안 스코프 제한 디테일:

- **IAMManageProjectResources**: Resource 를 `arn:aws:iam::*:role/project-*` 로 제한 → Jenkins 가 침해당해도 다른 IAM role 못 건드림
- **PassRoleScoped**: Condition `iam:PassedToService = ec2.amazonaws.com` → Lambda 등 다른 서비스로 권한 우회 차단 (privilege escalation 방어 표준 패턴)

5. 학습

5-1. 본인 학습 (Engineering)

- 🌸 **IaC 자기참조 구조의 함정**: 자기 권한을 자기가 관리하는 구조이면 **bootstrap** 경로를 별도로 설계 해야 함. 안 그러면 수정 불가능한 상태에 빠짐.
- 🌸 **POC vs Production IAM 패턴 차이**:

	POC (우리 사례)	Production
권한 범위	broad (<code>ec2:*</code> , <code>rds:*</code>)	최소권한
세션	static role	AssumeRole + 시간 제한
추적	사후 (CloudTrail)	사전 (Access Analyzer)

- 🌸 **Resource ARN prefix 제한** (`project-*`) 의 의미 = **blast radius 축소**. 침해 시 피해 범위 한정.
- 🌸 **PassRole Condition** 의 의미 = **privilege escalation 방어**. EC2 외 서비스로 권한 우회 차단.
- 🌸 **사전 완벽 최소권한 설계는 불가능** — Terraform provider 가 호출하는 API 가 수십 개. 실패 → 에러에서 action 확인 → 추가의 반복이 현실적.

5-2. Pre-Sales 학습 (클라우드 직무 관점)

고객사 IaC 도입 컨설팅 시 사전 설계 체크리스트:

1. Bootstrap path 분리 표준화

- "초기 권한 심기 (broad)" 와 "운영 권한 (제한)" 을 별도 IAM 계정/role 로 명확히 분리
- 운영 role 만으로는 자기 자신의 권한 변경 불가하도록 설계
- 운영 중 권한 추가 필요 시 bootstrap 경로로만 가능 → 의도하지 않은 권한 확장 차단

2. Production 진입 시 AssumeRole + 시간 제한 세션 패턴 전환

- broad permission 은 short-lived session 으로만 부여
- 일상 운영은 최소권한 role 로

3. Resource ARN prefix 제한 (`project-*`, `team-*`) 으로 blast radius 축소 의무화

4. PassRole Condition `iam:PassedToService` 로 privilege escalation 차단

5. **CloudTrail + IAM Access Analyzer** 로 실제 호출 추적 → 사후 데이터 기반 권한 축소가 현실적. "PoC 는 broad → Production 은 narrow" 패턴을 고객 runbook 에 표준 절차로.

→ 이 패턴이 표준화된 컨설팅 자산이 되면, 고객사 PoC → Production transition 의 보안 점검에서 **누락 항목 0건** 보장 가능.

Narrative 3: Ansible **ansible_user** 누락 — 자동화 환경의 암묵적 기본값 함정

1. 증상

💡 Phase 1 빌드 첫 시도, Ansible 이 첫 SSH 부터 실패:



```
on-prem-haproxy | UNREACHABLE!: jenkins@100.105.181.80: Permission denied
```

핵심 단서: **SSH 사용자**가 **ansible** 이 아니라 **jenkins** 로 시도됨.

2. 가설

💡 본인 첫 의심: "SSH key 가 잘못 등록됐나?" — 그런데 직접 SSH 는 통과했고, 에러 메시지에 사용자명이 **jenkins** 로 찍혀있는 게 이상함. 더 파야 함.



AI 가설 chain:

- 가설 A: SSH key 등록 문제 → 그러면 "publickey" 에러여야 하는데 이건 단순 **Permission denied** + 사용자 명이 **jenkins**. **에러 메시지의 사용자명이 이미 단서.**
- 가설 B: 사용자명이 어디서 잘못 결정되는가?
- 가설 C (정답): Ansible 의 connection 사용자 결정 메커니즘 —
 - inventory 에 **ansible_user** 명시되면 그 값
 - 명시 없으면 → **"현재 OS 사용자"**
 - Jenkins 는 **jenkins** 시스템 사용자로 돌아감
 - Ansible 이 **jenkins@호스트** 로 SSH 시도

3. 디버깅



Ansible 변수 적용 우선순위 안내 (낮 → 높):

- 기본값 (**현재 OS 사용자**) ← 여기 걸림
- group_vars/all.yml**
- group_vars/<group>.yml** (예: **webwas.yml**)
- host_vars/<hostname>.yml**
- 인벤토리 파일의 host 레벨
- 커맨드라인 **-u**

● 본인 inventory 검사: `ansible_user` 정의 어디에도 없음 확인 → 1번 (기본값) 적용 → `jenkins` 로 SSH 시도 가설 확정.

4. 해결

🤖 AI 추천: `group_vars/all.yml` 에 모든 호스트 공통 사용자 명시.

● 본인 적용:

```
# Ansible/inventories/on-premise-phase1/group_vars/all.yml
---
# SSH 연결 기본 사용자 (모든 호스트 공통)
ansible_user: ansible
timezone: Asia/Seoul
```

→ Phase 1 SSH 연결 정상화.

5. 학습

5-1. 본인 학습 (Engineering) — AI 진단을 본인 mental model 로 흡수

- 🤖 → ● "빈칸 = 기본값" 함정: AI 가 설명해준 메커니즘을 본인 toolkit 으로 흡수. 자동화 환경에서 가장 예상치 못한 동작은 **암묵적 기본값** 에서 발생:
 - Jenkins 같은 비대화형 환경
 - 시스템 사용자 (`jenkins`) 로 돌아가는 컨텍스트
 - 의도와 다른 기본값이 자연스럽게 적용됨
- 🤖 → ● 명시 박기 원칙: 자동화 관련 설정은 "*어차피 기본값이지*" 가 통하지 않음. `ansible_user`, `ansible_port`, `ansible_python_interpreter`, `ansible_become_method` 같은 connection 변수는 `group_vars` 에 명시.
- 에러 메시지를 의심하는 습관: 본인이 처음 "*키 문제인가?*" 로 가지 않고 "*근데 사용자명이 왜 jenkins 로 찍혀 있지?*" 라는 의문을 가졌던 게 시작점. **AI 진단도 결국 그 의문에서 출발했음** — 정확한 첫 의심이 디버깅 속도를 결정.

5-2. Pre-Sales 학습 (자동화 도입 컨설팅 관점)

고객사 Ansible 자동화 도입 시 표준 체크리스트:

1. **Inventory 표준 템플릿 강제** 모든 inventory 의 `group_vars/all.yml` 에 다음 4개 변수 필수 명시:

- `ansible_user`
- `ansible_port`
- `ansible_python_interpreter`
- `ansible_become_method`

2. **CI 단계에서 inventory linting** 위 변수 누락 시 PR merge 차단. 사람의 기억에 의존하지 않는 메커니즘으로.

3. **Ansible 실행 컨텍스트 사전 정의** 동일 playbook 이 다음 모든 컨텍스트에서 동일하게 작동하도록 설계:

- Jenkins 자동화 (jenkins 유저)
- 관리자 manual 실행 (개인 OS 유저)
- 동료 손익 (다른 OS 유저)

4. playbook 내부 첫 단계에 whoami debug task

```
- name: Verify connection identity
  debug:
    msg: "Running as {{ ansible_user_id }} on {{ inventory_hostname }}"
```

→ 어떤 유저로 동작하는지 즉시 확인 가능.

→ 이 4단계 표준화 시, 신규 환경 셋업의 첫 SSH 실패 디버깅이 **0회에 수렴**. 컨설팅 자산화하면 신규 고객 onboarding의 첫날 issue 한 종류 박멸.

발표 시 사용 톤 가이드

각 narrative 를 PPT [26] 슬라이드 1장으로 압축할 때:

- **슬라이드 본문:** 5단계 중 "증상 + 해결" 만 1-2줄씩 (스토리 보존)
- **슬라이드 하단 강조 박스:** Pre-Sales 학습의 한 줄 핵심 — "고객사 runbook 에 들어가야 할 사전 체크"
- **Speaker note:** 5단계 풀버전 + 본인 학습 (구두로 풀어서 설명)
- **본인 사고 vs AI 보조 구분:** 발표 중 "이 가설은 제가 의심했고, 이 명령어는 AI 도움을 받아 빠르게 도달했습니다" 식으로 정직하게 — **이게 청중에게 신뢰를 줍니다.** AI 시대의 엔지니어가 어떻게 일하는지를 보여주는 차별화 포인트.

Layer 분포 메시지 (3개 narrative 의 큰 그림)

Narrative	Layer	메시지
1. Tailscale ProxyJump → Subnet Router	Network	"디버깅보다 우회로 — 엔지니어링 판단력"
2. IAM Bootstrap 역설	Cloud Security	"IaC 자기참조 구조의 닭/달걀 — 클라우드 보안 설계 기본"
3. Ansible ansible_user 누락	Automation	"암묵적 기본값 함정 — 자동화 표준 체크리스트의 가치"

→ 발표 청중이 "DR 만이 아니라 클라우드 + 네트워크 + 자동화 전반의 사고력" 을 본다고 인식하게 됨.

백업 narrative — Q&A 대비 (슬라이드에 포함 X)

발표 중 "가장 어려웠던 디버깅 사례?" 같은 질문 들어오면 꺼낼 수 있는 추가 narrative. 슬라이드에는 안 넣고 본인 머릿속 + 이 문서에만 보유.

Backup: GTID Multi-Node Align (04-26 §4 사례 6)

한 줄 요약: Cascade replication 의 RDS 단계에서 "*Slave_IO_Running: Yes* 인데 *Retrieved_Gtid_Set* 이 빈 값" — 4시간 디버깅 끝에 RDS 가 가짜 GTID 1-18 을 누적하고 있었던 게 원인.

왜 정점 narrative 인가:

- 🗨️ "*Plan B (cascade 우회) 거부, 끝까지 가자*" 는 결정 — 본인 사고의 정수
- 🗨️ mismatch matrix 비교가 결정적 단서 — multi-node distributed system 의 가시성 (observability) 통찰
- 🚗 가설 후보 빠르게 제공, 결정과 판단은 본인 — AI 협업의 모범 사례

Q&A 활용 시 강조 포인트:

- AWS RDS 같은 managed service 의 SUPER 권한 차단 → vanilla mysql 의 *RESET MASTER, SET GLOBAL gtid_purged* 같은 도구 사용 불가 → 비표준 우회 (empty transactions 으로 GTID align) 학습
- "*managed service* 는 자유도 ↓ *trade-off* 가 있다" 는 클라우드 직무 사고

상세는 [dr-project-review-2026-04-26-day-summary.md](#) §4 사례 6.